Name: Om Rajale
AIEC 2
Roll 26
Batch A

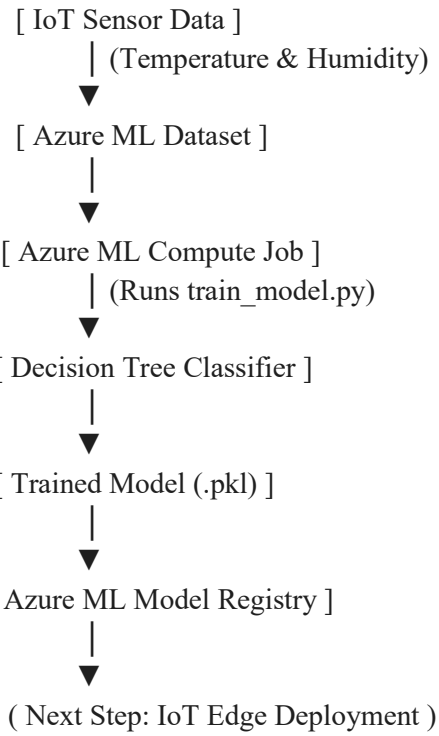
Experiment – 07
Building an Azure Machine Learning Model for an IoT Edge Module

Objective: Learn how to collect IoT sensor data, prepare a dataset, train a Machine Learning model in Azure ML, register the trained model, and prepare it for deployment to an IoT Edge module.

1. Overview & Architecture Workflow

In this lab, we will train a **Decision Tree Classifier** using simulated IoT sensor data (Temperature and Humidity) to predict device status (Normal vs. High).

Workflow Diagram



Step 1 — Open Azure Machine Learning Studio

1. Navigate to Azure Machine Learning Studio.

2. Sign in using your Azure credentials.
3. Select your designated workspace

Step 2 — Create an Azure ML Workspace

If a workspace does not already exist, create one via the **Azure Portal**:

1. Search for **Machine Learning** in the search bar.
2. Click **Create** > **Machine Learning**.
3. Configure the following properties:

Setting	Example Value
Subscription	*Your Azure Subscription*
Resource Group	IOTEdgelabRG
Workspace Name	aml-iot-lab
Region	Central India

4. Click **Review + Create** → **Create**. Once created, launch Azure ML Studio.

Step 3 — Create the Training Dataset

We will create a CSV file named iot_temperature_humidity.csv.

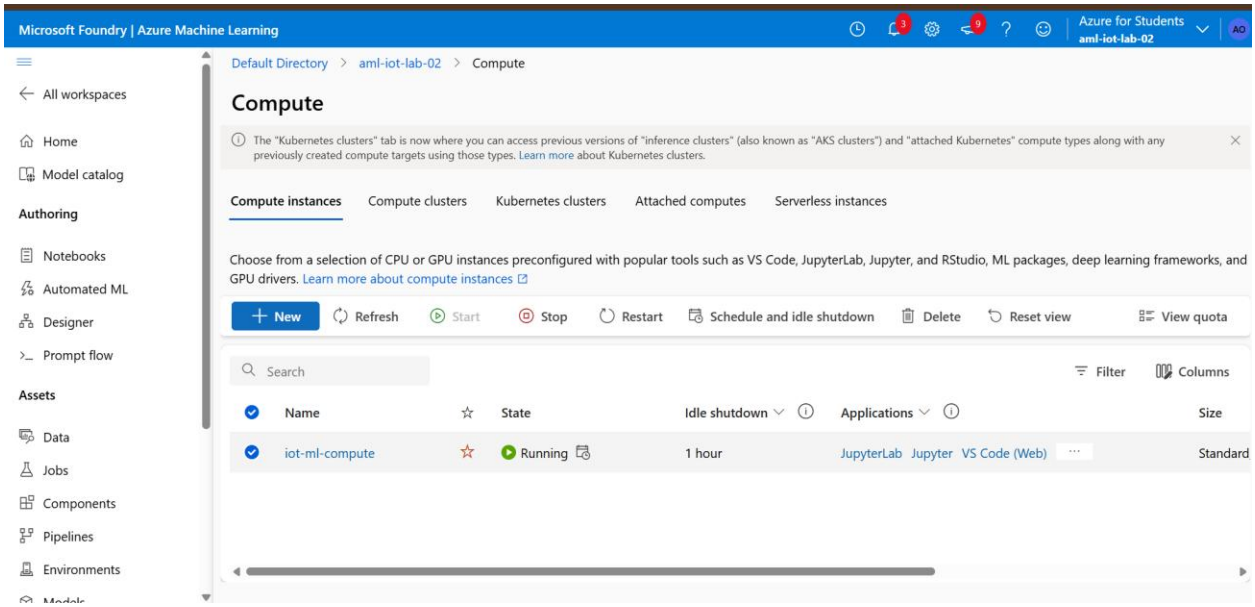
CSV Content

temperature,humidity,status
22,45,Normal
24,50,Normal
25,55,Normal
27,60,Normal
29,65,Normal
31,70,High
33,75,High
35,80,High
20,40,Normal
23,48,Normal
26,52,Normal
28,58,Normal
30,68,High

32,72,High
34,78,High
21,42,Normal
25,50,Normal
Dataset Structure

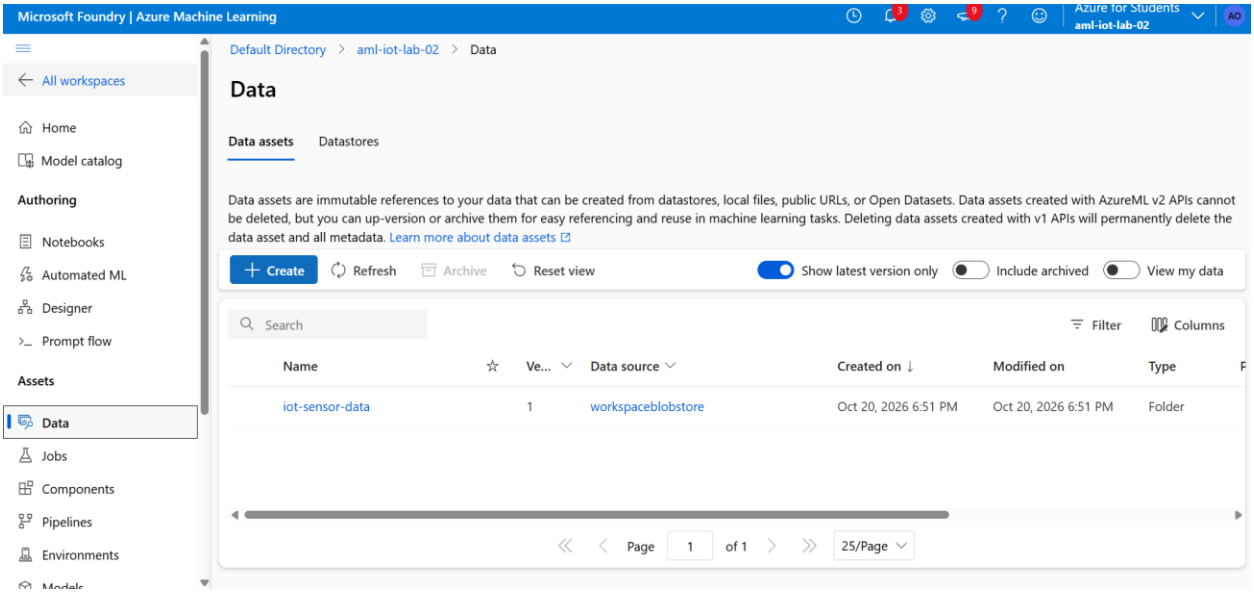
- **temperature**: Input feature (IoT telemetry variable 1)
- **humidity**: Input feature (IoT telemetry variable 2)
- **status**: Target label (Normal or High)

Step 4 — Create a Compute Instance

1. In Azure ML Studio, click **Compute** from the left side menu.
2. Under **Compute instances**, click **New**.
3. Choose a low-cost virtual machine size (e.g., Standard_DS11_v2).
4. Set the compute name to: iot-ml-compute.
5. Click **Create** and wait until the state updates to **Running**.
6. 

Step 5 — Upload Data Asset to Azure ML

1. Go to **Data** → **Create** → **Data asset**.
2. Select **From local files**.
3. Upload iot_temperature_humidity.csv.
4. Name the asset**: iot-sensor-data.**
5. Set the type to **Tabular** and complete the wizard.



Step 6 — Create the Python Training Script (train_model.py)

Create a script titled train_model.py:

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Load dataset
data = pd.read_csv("iot_temperature_humidity.csv")

# 2. Define input features (X) and target label (y)
X = data[["temperature", "humidity"]]
y = data["status"]

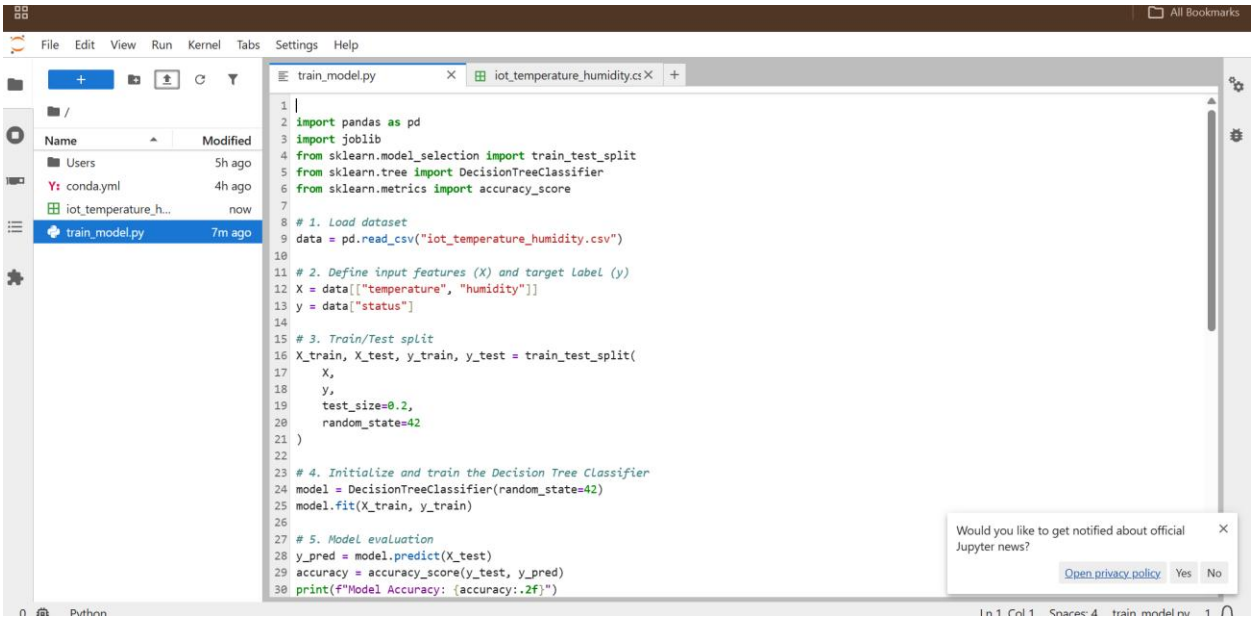
# 3. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# 4. Initialize and train the Decision Tree Classifier
```

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# 5. Model evaluation
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")

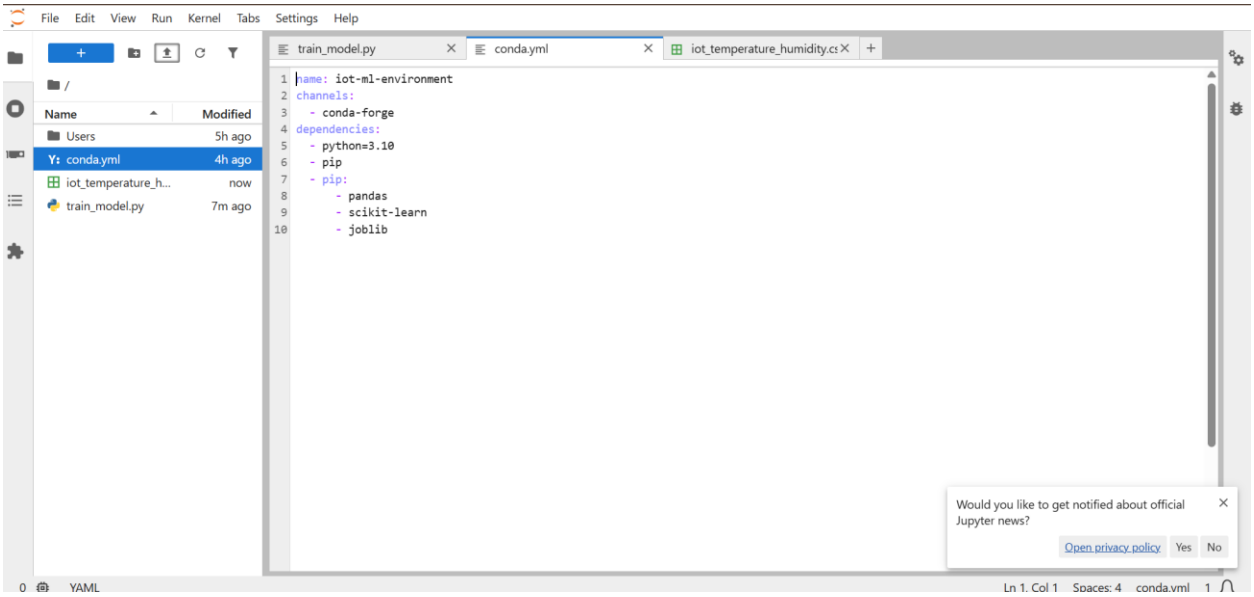
# 6. Export the serialized model file
joblib.dump(model, "iot_temperature_model.pkl")
print("Model saved successfully as iot_temperature_model.pkl")
```



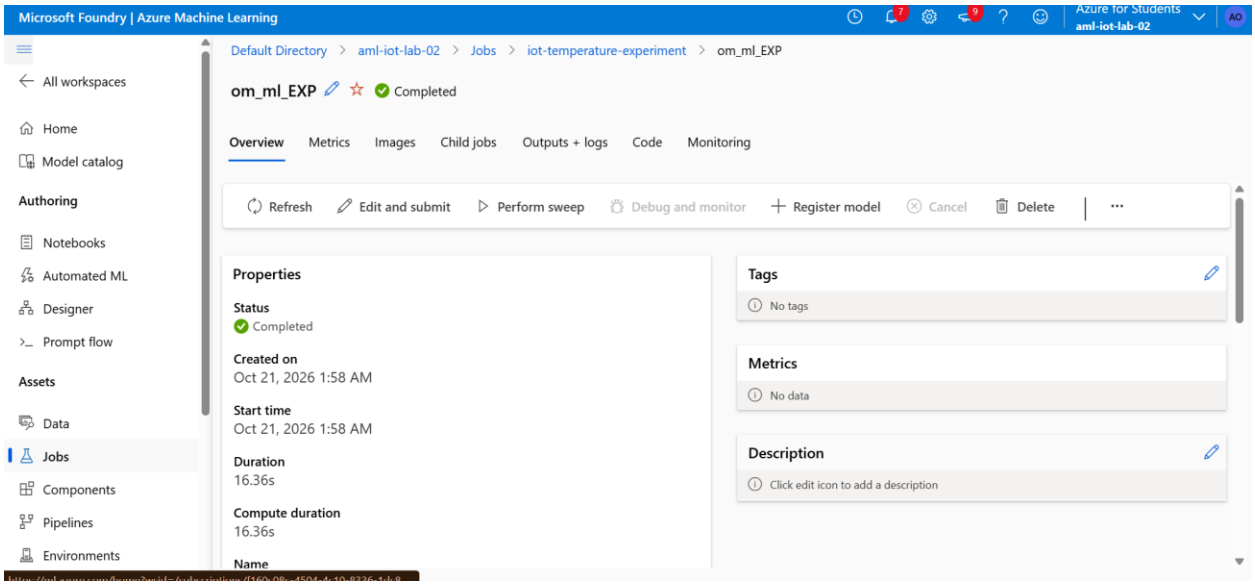
Step 7 — Define the Environment (conda.yml)

Create a configuration file named conda.yml to specify runtime dependencies:

```
name: iot-ml-environment
channels:
  - conda-forge
dependencies:
  - python=3.10
  - pip
  - pip:
      - pandas
      - scikit-learn
      - joblib
```

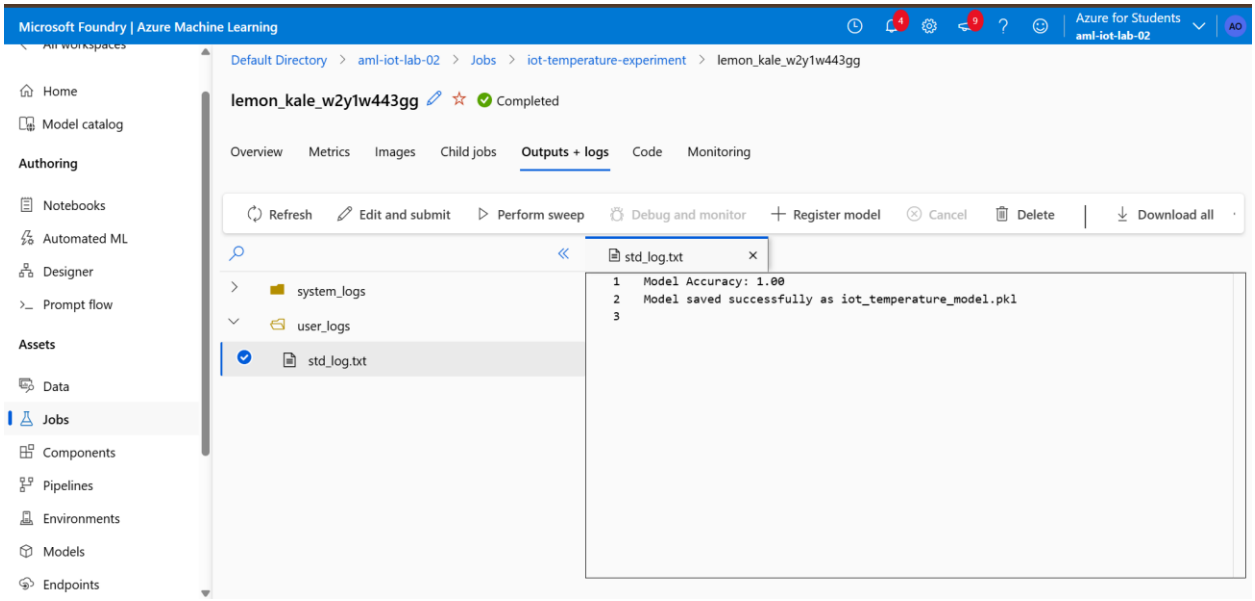


Step 8 — Submit and Run the Training Job

1. In Azure ML Studio, go to **Jobs** → **Create job**.
2. Select **ML Job**.
3. Set the execution command:
   python train_model.py
4. Attach train_model.py and iot_temperature_humidity.csv.
5. Target Compute: iot-ml-compute.
6. Set the Environment using conda.yml.
7. Click **Submit**.
8. 

Step 9 — Evaluate Model Execution & Accuracy

1. Open the completed Job in Azure ML Studio.
2. Select **Outputs + logs** and review the execution logs (std_log.txt).
3. **Verify output:**
   Model Accuracy: 0.75
   Model saved successfully as iot_temperature_model.pkl



Step 10 — Register Model in Registry

1. Navigate to **Models** → **Register** → **From local files**.
2. Upload iot_temperature_model.pkl.
3. Set Name: iot-temperature-model.
4. Set Version: 1.
5. Click **Register**.

Step 11 — Verify the Registered Model

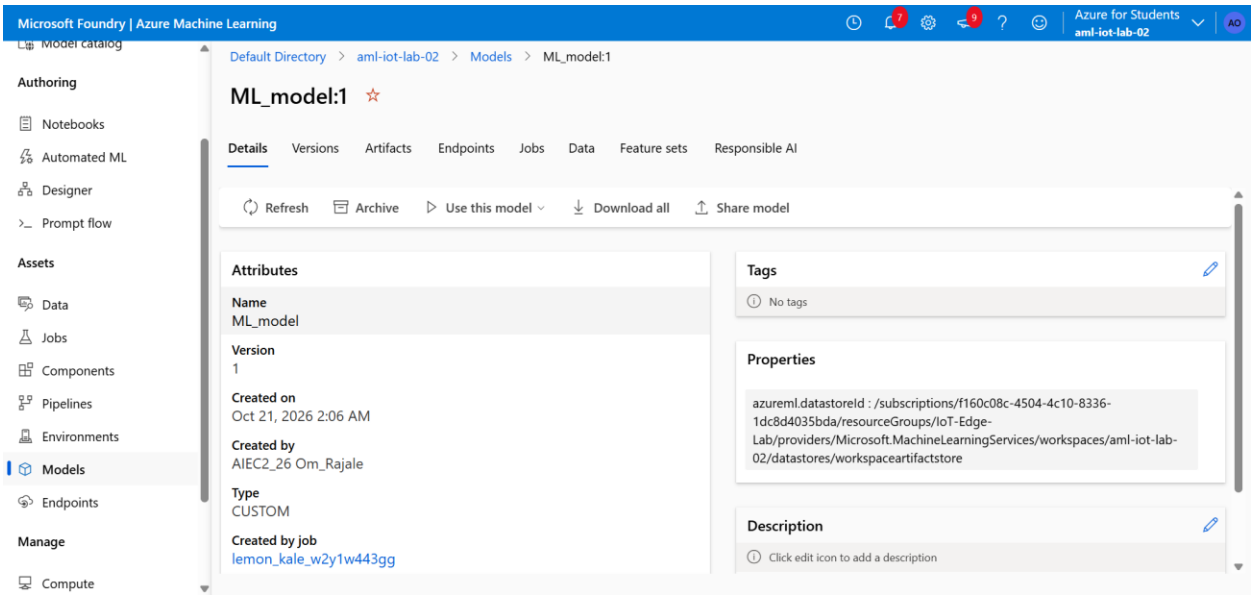
Go to:

Models

iot-temperature-model

with a version such as: 1

This is the trained ML model that we can use for the next practical.



Step 12 — Model Inference Logic (Sample Tests)

Scenario	Input (Temperature, Humidity)	Expected Model Prediction
Case 1	$Temp =$, $Humidity =$	**High**
Case 2	$Temp =$, $Humidity =$	**Normal**

Summary Checklist

- Created Azure ML Workspace & Compute Instance
- Prepared IoT Telemetry CSV Data Asset
- Written train_model.py and environment definition (conda.yml)
- Executed Azure ML Job and verified accuracy
- Registered iot-temperature-model:1 for IoT Edge container deployment